

Course Materials for NLP Term Project: BPE and WordPiece Tokenization: A Worked Example for Subword Tokenizers

Prof. Moon-Ryul Jung
Dept of Art & Technology
Sogang University

1 Introduction

This tutorial traces **Byte-Pair Encoding (BPE)** and **WordPiece** on a small worked example. These are two of the three main subword tokenization algorithms used in modern NLP (the third being UnigramLM, covered in a separate tutorial).

Both BPE and WordPiece are **bottom-up, greedy, merge-based** algorithms. They start from a vocabulary of single characters and iteratively merge useful adjacent pairs into new tokens. The two differ in two important ways:

- **Merge criterion:** BPE [1] merges the pair with the highest *co-occurrence frequency*; WordPiece [2, 3] merges the pair with the highest *pointwise mutual information* (PMI) score.
- **Word-position handling:** BPE treats every character the same regardless of its position in a word; WordPiece distinguishes *word-initial* characters from *word-internal* characters, prepending `##` to the latter and treating them as different tokens.

The second difference is often glossed over in introductory expositions but it is real and consequential: it changes the initial vocabulary, the pair counts, the PMI scores, and ultimately the merges that get selected. We will trace both algorithms carefully, keeping this distinction visible at every step.

2 Which Models Use What

Note that **SentencePiece** is a **library, not an algorithm**. SentencePiece can run either BPE or UnigramLM depending on the `model_type` flag.

3 Setup: The Training Corpus

We use the same small training corpus throughout:

$$\mathcal{W} = \{(\text{low}, 5), (\text{lower}, 2), (\text{newest}, 6), (\text{widest}, 3)\} \quad (1)$$

These are pre-tokens (whitespace-separated words) with their frequencies. The total corpus has $5 + 2 + 6 + 3 = 16$ pre-token occurrences.

Model	Algorithm	Notes
GPT-2, GPT-3, GPT-4	BPE (byte-level)	Operates on raw bytes, not Unicode characters
RoBERTa	BPE (byte-level)	Inherits from GPT-2
LLaMA, LLaMA-2, LLaMA-3	BPE (via SentencePiece)	<code>model_type=bpe</code> in SentencePiece
Mistral	BPE (via SentencePiece)	Same as LLaMA
BERT, DistilBERT	WordPiece	Original Google WordPiece
ELECTRA	WordPiece	Inherits from BERT
mBERT	WordPiece	Multilingual BERT, WordPiece on 104 languages
XLNet, mT5, NLLB, Glot500	UnigramLM	via SentencePiece, <code>model_type=unigram</code>

Table 1: Tokenization algorithm by model.

3.1 Word-Position Analysis

Before defining the initial vocabularies, we analyze where each character appears in this corpus. This will matter for WordPiece.

Character	Word-initial positions	Word-internal positions	Total
l	low (5), lower (2) → 7	— → 0	7
o	— → 0	low (5), lower (2) → 7	7
w	widest (3) → 3	low (5), lower (2), newest (6) → 13	16
e	— → 0	lower (2), newest (12), widest (3) → 17	17
r	— → 0	lower (2) → 2	2
n	newest (6) → 6	— → 0	6
s	— → 0	newest (6), widest (3) → 9	9
t	— → 0	newest (6), widest (3) → 9	9
i	— → 0	widest (3) → 3	3
d	— → 0	widest (3) → 3	3

Table 2: Character occurrence counts by word position. Note that “newest” contributes *twice* to e word-internally (positions 1 and 3), so $6 \times 2 = 12$.

Three observations from this analysis:

1. Characters l, n appear **only word-initially** in this corpus.
2. Characters o, e, r, s, t, i, d appear **only word-internally**.
3. Character w appears **in both positions**: word-initially in “widest” and word-internally in “low”, “lower”, “newest”.

4 BPE’s Initial Vocabulary V_0^{BPE}

BPE treats every character the same regardless of position. The initial vocabulary is simply the set of distinct characters:

$$V_0^{\text{BPE}} = \{l, o, w, e, r, n, s, t, i, d\} \quad (2)$$

So $|V_0^{\text{BPE}}| = 10$. Each character is one token regardless of where it appears.

Initial corpus segmentation for BPE:

Pre-token	Frequency	BPE initial segmentation
low	5	(l, o, w)
lower	2	(l, o, w, e, r)
newest	6	(n, e, w, e, s, t)
widest	3	(w, i, d, e, s, t)

Table 3: Initial corpus segmentation under BPE.

5 WordPiece’s Initial Vocabulary V_0^{WP}

WordPiece distinguishes word-initial from word-internal characters. The **##** prefix marks word-internal tokens. A character only enters the vocabulary in a form (with or without **##**) if it actually appears in the corresponding position.

From the word-position analysis above:

- Word-initial-only characters $\rightarrow$ enter V_0^{WP} **without ##**: $\{l, n\}$.
- Word-internal-only characters $\rightarrow$ enter V_0^{WP} **with ##**: $\{##o, ##w, ##e, ##r, ##s, ##t, ##i, ##d\}$.
- Character w appears in both positions $\rightarrow$ enters **both** forms: $\{w, ##w\}$.

Therefore:

$$V_0^{\text{WP}} = \{ \underbrace{l, n, w}_{\text{word-initial}}, \underbrace{##o, ##w, ##e, ##r, ##s, ##t, ##i, ##d}_{\text{word-internal}} \} \quad (3)$$

So $|V_0^{\text{WP}}| = 11$. The word-initial w and word-internal **##w** are *different tokens* as far as WordPiece is concerned. They have different embeddings (in any model using this tokenizer) and they participate in different pair-merge decisions.

Initial corpus segmentation for WordPiece:

Pre-token	Frequency	WordPiece initial segmentation
low	5	(l, ##o, ##w)
lower	2	(l, ##o, ##w, ##e, ##r)
newest	6	(n, ##e, ##w, ##e, ##s, ##t)
widest	3	(w, ##i, ##d, ##e, ##s, ##t)

Table 4: Initial corpus segmentation under WordPiece. The first character of each word has no **##**; subsequent characters carry **##**. The w in “widest” is bare while the **##w** in the other words has the prefix.

5.1 Comparison of Initial Vocabularies

WordPiece starts with one more token than BPE here because w appears in both word-initial and word-internal positions, giving rise to two distinct tokens w and **##w**. In a larger corpus with many characters appearing in both positions, this difference grows: for example, if every one of the 26 English letters appeared in both positions, WordPiece’s V_0 would have 52 tokens to BPE’s 26.

Algorithm	$ V_0 $	Initial vocabulary
BPE	10	{l, o, w, e, r, n, s, t, i, d}
WordPiece	11	{l, n, w, ##o, ##w, ##e, ##r, ##s, ##t, ##i, ##d}

Table 5: Initial vocabularies of BPE and WordPiece on the same corpus.

6 The Training Budget: What is V_{target} ?

Before describing how BPE and WordPiece run, we need to understand the most important hyperparameter of a tokenizer: the **target vocabulary size**, written V_{target} . This is the user-specified number of tokens that the final tokenizer will know. Choosing this number well is one of the most consequential decisions in training a tokenizer, so it is worth spending some time on what it means and how to think about it.

6.1 Why we need a budget at all

A subword tokenizer’s vocabulary is a finite list of strings (the “tokens”) that the tokenizer can output. Every input word will be broken into a sequence of these tokens. The vocabulary has two competing pressures:

Too small. If the vocabulary contains only single characters (say 26 letters of English), every word becomes a long sequence of characters. The word “newest” becomes 6 tokens: (n, e, w, e, s, t). This makes the sequences fed into the model long, slowing down both training and inference. It also forces the model to do extra work to learn that the character sequence e, s, t usually means the suffix “-est”.

Too large. If the vocabulary contains every word that ever appears (say a million distinct words), every word maps to one token. “newest” becomes (newest), a single token. This is fast at inference, but the embedding matrix — the lookup table that maps each token to its dense vector representation — becomes huge. The model has to learn a separate embedding for every word, which requires lots of training data per word. And rare words have so little data that their embeddings remain poorly learned. Worse, a word never seen during training becomes an unknown token [UNK].

Subword tokenization is the middle ground. It builds a vocabulary that includes common whole words (like “the”, “of”, “low”) AND useful sub-word pieces (like “-est”, “-ing”, “un-”) AND single characters as a fallback for anything else. This lets the tokenizer balance:

- **Compression:** most words become 1–3 tokens rather than character-by-character.
- **Coverage:** any word, even one never seen in training, can be tokenized (it might just take more tokens).
- **Learnable embeddings:** rare words share embeddings via their common subword pieces (e.g., “newest” shares “est” with “widest”, “oldest”, etc., so the meaning of the suffix is learned from many words).

The target vocabulary size V_{target} controls where on this spectrum the tokenizer sits. A larger V_{target} means longer multi-character tokens, shorter token sequences, but bigger embedding matrix. A smaller V_{target} means more character-level segmentations, longer sequences, but smaller embedding matrix.

6.2 Typical vocabulary sizes in practice

To calibrate intuition, here are the actual vocabulary sizes of the models discussed earlier:

Model	Vocab size	Notes
BERT (English)	30,522	WordPiece on English Wikipedia + BookCorpus
mBERT (104 languages)	119,547	WordPiece on Wikipedia in 104 languages
GPT-2	50,257	Byte-level BPE on WebText
GPT-4 (c1100k_base)	100,000	Byte-level BPE
LLaMA-2	32,000	BPE via SentencePiece on multilingual web
LLaMA-3	128,000	BPE via SentencePiece, larger for multilingual
XLNet (100 languages)	250,000	UnigramLM via SentencePiece
Glottos (511 languages)	401,145	UnigramLM, extends XLNet's 250K + 151K new
NLLB (200 languages)	256,206	UnigramLM via SentencePiece

Table 6: Vocabulary sizes of well-known models.

A few patterns emerge:

- **Monolingual English models:** 30K–50K tokens. This is enough to cover most common English words as single tokens plus subword pieces for the rare ones.
- **Multilingual models:** 100K–400K tokens. More languages means more characters, more morphological patterns, and more whole words to cover.
- **Recent models (LLaMA-3, GPT-4):** 100K+ tokens. Modern models have larger vocabularies because larger embedding matrices became affordable as overall model sizes grew into the billions of parameters.

For our small worked example, we use $V_{\text{target}} = 14$ — absurdly small, but small enough that we can trace every algorithmic step by hand. The principles are the same as for 250K tokens; only the corpus and the iteration count change.

6.3 From vocabulary size to iteration count

For **bottom-up algorithms** (BPE and WordPiece), the relationship between vocabulary size and training is direct: each iteration adds exactly one token to the vocabulary. So the number of iterations is:

$$N_{\text{iter}} = V_{\text{target}} - |V_0| \quad (4)$$

where $|V_0|$ is the size of the initial vocabulary (the single-character vocabulary, before any merges). This is a deterministic relationship — you can compute the number of iterations the moment you decide V_{target} and know your character set.

For **top-down algorithms** (UnigramLM), the relationship is different: training starts from a large seed vocabulary V_{init} and prunes it down to V_{target} . The number of pruning iterations depends on the shrinkage factor η (typically $\eta \approx 0.8$): each iteration removes roughly $(1 - \eta) = 20\%$ of the remaining tokens, so the number of iterations is approximately

$$N_{\text{iter}} \approx \frac{\log(V_{\text{target}}/V_{\text{init}})}{\log \eta} \quad (5)$$

For example, with $V_{\text{init}} = 10^6$ and $V_{\text{target}} = 30,000$, this comes to about 16 pruning iterations (each interleaved with an EM loop). The UnigramLM tutorial covers this in detail; here we focus on a specific implementation question that often confuses newcomers — how the seed vocabulary V_{init} is actually constructed and stored.

6.4 How V_{init} is Constructed (Implicitly to the User)

UnigramLM has a seed vocabulary V_{init} that is much larger than the final V_{target} . From the user's perspective this seed is implicit: when invoking SentencePiece, you specify only `-vocab_size=V_target` and the library takes care of the rest. But internally, V_{init} is constructed by an explicit deterministic procedure with several steps.

Step 1 — Enumerate frequent substrings via suffix arrays. The library scans the corpus and finds all distinct substrings of pre-tokens up to a maximum length controlled by `-max_sentencepiece_length` (default 16 characters). To do this efficiently — naive enumeration would be $O(n^2)$ in corpus size — SentencePiece uses an **Enhanced Suffix Array (ESA)**, which can identify all repeated substrings in roughly linear time along with their exact occurrence counts.

Step 2 — Score and select candidates. Each candidate substring is scored by a heuristic combining its length and corpus frequency (longer + more frequent = higher score). The top K substrings are taken as candidates, where K is controlled by `-seed_sentencepiece_size` (default $K = 1,000,000$).

Step 3 — Ensure character coverage. The candidate set is augmented with single characters from the corpus to guarantee that every character can be tokenized. The `-character_coverage` flag (default 0.9995, meaning 99.95% of characters by frequency) determines how aggressive this is. The rare 0.05% of characters might be dropped, with their handling deferred to byte-fallback tokens at inference.

Step 4 — Add control tokens. Special tokens like `<unk>`, `<s>`, `</s>`, `<pad>` are added, along with any user-specified additional special tokens.

After these four steps, V_{init} is fixed and the UnigramLM pruning iterations begin: run EM to estimate token probabilities, prune the bottom $1 - \eta$ fraction of tokens by likelihood contribution, repeat until $|V| \leq V_{\text{target}}$.

So V_{init} is implicit only in the user-facing API sense (you do not pass it as an argument). The procedure that constructs it is fully explicit and deterministic, and the user can override the defaults via flags like `-seed_sentencepiece_size`, `-max_sentencepiece_length`, and `-character_coverage` if needed.

6.5 Is V_{init} Stored Explicitly? (Yes)

A natural follow-up question: when SentencePiece builds V_{init} with about a million tokens, is this stored as a concrete data structure in memory, or is it represented implicitly (e.g., by a procedure that re-derives substrings on demand)?

The answer is: explicitly stored as a concrete data structure. In SentencePiece's C++ implementation (the file `src/unigram_model_trainer.cc` in the SentencePiece repository), the seed vocabulary lives as an `std::vector<std::pair<std::string, double>>` — a vector of (token string, log-probability) pairs. Each entry holds the actual UTF-8 string of the candidate token (e.g., `"_super"`, `"ing"`, `"de"`, `"a"`) and an initial log-probability used to start the EM iterations.

Memory footprint. For a million tokens of average length about 5 characters (a reasonable estimate when most tokens are short subwords with a long tail of longer ones), the memory cost is roughly:

- Token strings: $10^6 \times 5$ bytes (UTF-8) = 5 MB, plus `std::string` overhead (~ 24 bytes per string on 64-bit systems), bringing the total to ~ 30 MB.
- Log-probabilities: $10^6 \times 8$ bytes (double) = 8 MB.
- Vector overhead: negligible.

Total: around **40 MB** for the seed vocabulary. For very large corpora with `-seed_sentencepiece_size=500000` (5 million seeds), the footprint grows to about 200 MB. Both are trivial by modern standards — easily fit in RAM on any laptop, let alone a training server.

Auxiliary data structures. The seed vector is not the only thing in memory during training. To efficiently enumerate which tokens in V can segment a given pre-token, SentencePiece builds a **trie** (or sometimes a DAG) over the strings in V . Given a pre-token like “newest”, the trie can quickly tell you which tokens in V are prefixes of “newest” starting at each position. The trie is rebuilt or updated each time V changes (after each pruning step). A **token-to-index hash map** is also maintained for $O(1)$ lookup of a token’s index in the vector. During EM, transient per-pre-token expected-count statistics are accumulated and then discarded each iteration. Total memory across all these structures in a typical training run is on the order of 100–500 MB, dominated by the trie and the cached EM statistics.

Why explicit storage is the natural choice. You might ask why the seed isn’t computed lazily on demand. Three reasons:

- The EM step needs random access to token probabilities; flat vector with $O(1)$ index access is much faster than re-deriving substrings from the corpus on every EM step.
- The segmentation trie needs the actual token strings, not hashes or corpus offsets.
- Memory is cheap relative to compute; optimizing for memory at the cost of EM speed would be a bad tradeoff at this scale.

After training, V_{init} is discarded. Once UnigramLM training finishes and $|V| = V_{\text{target}}$, only the final vocabulary is saved to the `.model` file. The pruned-away candidates (the difference between V_{init} and V_{target} , often a million tokens for $V_{\text{target}} = 30,000$) are discarded. The on-disk SentencePiece model file contains only the final V_{target} tokens, their final log-probabilities (used at inference for Viterbi), special tokens, and configuration. A typical `.model` file for $V_{\text{target}} = 32,000$ is about 500 KB to 1 MB on disk.

Practical implication for the Coptic-Syriac project. When you train SentencePiece UnigramLM on your Coptic + Syriac + Glot500-c sample to discover new tokens, expect peak memory usage of about 200–500 MB during training, mostly from the seed vocabulary and EM statistics. Training is CPU-bound, not GPU-bound — SentencePiece does not use the GPU. So tokenizer training can happen on any machine with enough RAM, and the resulting `.model` file can then be moved to your A6000 for the actual continued pretraining of Glot500-m.

6.6 Iteration counts for our example

We target $V_{\text{target}} = 14$ for both BPE and WordPiece. Since each algorithm starts at a different $|V_0|$, they need different numbers of merge iterations:

- **BPE:** starts with $|V_0^{\text{BPE}}| = 10$ characters. Needs $14 - 10 = 4$ merge iterations to reach V_{target} .
- **WordPiece:** starts with $|V_0^{\text{WP}}| = 11$ tokens (because `w` appears in both positions, so `w` and `##w` are separate tokens). Needs $14 - 11 = 3$ merge iterations.

This is another corpus-dependent asymmetry. WordPiece’s **##** distinction causes its initial vocabulary to be larger whenever any character appears in both word-initial and word-internal positions. As a result, WordPiece has fewer iteration “budget” for building multi-character tokens for the same V_{target} .

On our tiny corpus this difference (4 vs 3 iterations) substantially changes the final vocabularies. On real corpora with V_{target} in the tens or hundreds of thousands, this difference becomes negligible — both algorithms have ample iteration budget. But the underlying source of the asymmetry (the **##** distinction inflating $|V_0|$) remains, just spread over many more iterations.

6.7 What “training” actually means here

Worth flagging one terminological note: when we say BPE or WordPiece is “trained” on a corpus, we do not mean gradient-based training with backpropagation. There are no neural network weights to update. Training a tokenizer means **running a deterministic algorithm** that scans the corpus, counts statistics, and decides which merges to perform. The output is a vocabulary (and for BPE, an ordered list of merge rules). This output is a fixed structure that is then used at inference time to tokenize new text.

So “training a BPE tokenizer with $V_{\text{target}} = 14$ on this corpus” is shorthand for “run the BPE algorithm on this corpus until the vocabulary reaches 14 tokens”. The training is deterministic given the corpus, V_{target} , and the tie-breaking rule. Re-running it on the same corpus with the same hyperparameters produces an identical tokenizer.

7 Part I: BPE Algorithm

7.1 The BPE Merge Criterion

At each iteration, BPE counts the **co-occurrence frequency** of every adjacent pair of tokens in the corpus. The pair with the highest co-occurrence frequency is merged into a new token, which is added to the vocabulary.

Formally, for each adjacent pair (a, b) in the corpus, define:

$$\text{freq}(a, b) = \sum_i c_i \cdot \#_i(a, b) \quad (6)$$

where c_i is the frequency of pre-token w_i and $\#_i(a, b)$ is the number of times tokens a and b appear adjacent in the current segmentation of w_i . The merge rule chosen at iteration t is:

$$(a^*, b^*) = \arg \max_{(a, b)} \text{freq}(a, b) \quad (7)$$

A new token a^*b^* is added to V_t , and the corpus segmentation is updated by replacing every occurrence of the adjacent pair (a^*, b^*) with the merged token.

7.2 BPE Iteration 1

Pair counts in the initial BPE segmentation:

Two pairs tie at 9: (e, s) and (s, t). Lexicographic tie-breaking picks (e, s).

Merge: $e + s \rightarrow es$. $V_1^{\text{BPE}} = V_0^{\text{BPE}} \cup \{es\}$. Merge rule $\mathcal{R}_1 : (e, s) \rightarrow es$.

Updated segmentations:

Pair	From	Count contribution	Total
(l, o)	low (5), lower (2)	$5 + 2$	7
(o, w)	low (5), lower (2)	$5 + 2$	7
(w, e)	lower (2), newest (6)	$2 + 6$	8
(e, r)	lower (2)	2	2
(n, e)	newest (6)	6	6
(e, s)	newest (6), widest (3)	$6 + 3$	9
(s, t)	newest (6), widest (3)	$6 + 3$	9
(w, i)	widest (3)	3	3
(i, d)	widest (3)	3	3
(d, e)	widest (3)	3	3

Table 7: Pair co-occurrence counts at BPE iteration 1.

Pre-token	Freq	Segmentation after merge 1
low	5	(l, o, w)
lower	2	(l, o, w, e, r)
newest	6	(n, e, w, es, t)
widest	3	(w, i, d, es, t)

7.3 BPE Iteration 2

Pair	From	Count contribution	Total
(l, o)	low, lower	$5 + 2$	7
(o, w)	low, lower	$5 + 2$	7
(w, e)	lower	2	2
(e, r)	lower	2	2
(n, e)	newest	6	6
(w, es)	newest	6	6
(es, t)	newest, widest	$6 + 3$	9
(w, i)	widest	3	3
(i, d)	widest	3	3
(d, es)	widest	3	3

Highest is (es, t) at 9.

Merge: $es + t \rightarrow est$. $V_2^{\text{BPE}} = V_1^{\text{BPE}} \cup \{est\}$. Rule $\mathcal{R}_2 : (es, t) \rightarrow est$.

7.4 BPE Iteration 3

After updating segmentations, pair counts:

Tie between (l, o) and (o, w) at 7. Lexicographic tie-breaking picks (l, o).

Merge: $l + o \rightarrow lo$. $V_3^{\text{BPE}} = V_2^{\text{BPE}} \cup \{lo\}$. Rule $\mathcal{R}_3 : (l, o) \rightarrow lo$.

7.5 BPE Iteration 4

Highest is (lo, w) at 7.

Merge: $lo + w \rightarrow low$. $V_4^{\text{BPE}} = V_3^{\text{BPE}} \cup \{low\}$, so $|V_4^{\text{BPE}}| = 14 = V_{\text{target}}$. Training terminates.

Pair	From	Count	Total
(l, o)	low, lower	5 + 2	7
(o, w)	low, lower	5 + 2	7
(w, e)	lower	2	2
(e, r)	lower	2	2
(n, e)	newest	6	6
(w, est)	newest	6	6
(w, i)	widest	3	3
(i, d)	widest	3	3
(d, est)	widest	3	3

Pair	From	Count	Total
(lo, w)	low, lower	5 + 2	7
(w, e)	lower	2	2
(e, r)	lower	2	2
(n, e)	newest	6	6
(w, est)	newest	6	6
(w, i)	widest	3	3
(i, d)	widest	3	3
(d, est)	widest	3	3

7.6 Final BPE Vocabulary and Rules

$$V^{\text{BPE}} = \{l, o, w, e, r, n, s, t, i, d, es, est, lo, low\} \quad (8)$$

Merge rules in order:

$$\mathcal{R}^{\text{BPE}} = \begin{cases} \mathcal{R}_1 : (e, s) \rightarrow es \\ \mathcal{R}_2 : (es, t) \rightarrow est \\ \mathcal{R}_3 : (l, o) \rightarrow lo \\ \mathcal{R}_4 : (lo, w) \rightarrow low \end{cases} \quad (9)$$

7.7 BPE Tokenization at Inference

“newest”:

1. Start: (n, e, w, e, s, t).
2. Apply $\mathcal{R}_1 : (e, s) \rightarrow es$. Match at positions 4-5. Result: (n, e, w, es, t).
3. Apply $\mathcal{R}_2 : (es, t) \rightarrow est$. Match at 4-5. Result: (n, e, w, est).
4. Apply $\mathcal{R}_3, \mathcal{R}_4$: no match.

Final: $\tau_{\text{newest}}^{\text{BPE}} = (n, e, w, est)$

“low”: After applying rules, $\tau_{\text{low}}^{\text{BPE}} = (low)$.

“newer” (out-of-vocabulary): No rule fires. $\tau_{\text{newer}}^{\text{BPE}} = (n, e, w, e, r)$.

8 Part II: WordPiece Algorithm

8.1 The WordPiece Merge Criterion

WordPiece chooses the pair with the highest **PMI(pointwise mutual information) score**:

$$\text{score}(a, b) = \text{PMI}(a, b) = \frac{\text{freq}(a, b)}{\text{freq}(a) \cdot \text{freq}(b)} \quad (10)$$

where $\text{freq}(t)$ is the corpus frequency of token t in the current segmentation and $\text{freq}(a, b)$ is the co-occurrence frequency of adjacent a and b .

Concrete example. Consider a corpus where:

- $P(\text{"New"}) = 0.001$ (the word “New” appears in 0.1% of word slots).
- $P(\text{"York"}) = 0.0008$ (the word “York” appears in 0.08% of word slots).
- $P(\text{"New York"}) = 0.0007$ (the bigram “New York” appears in 0.07% of bigram slots).

If “New” and “York” were independent, we would expect

$$P(\text{"New York"}) = P(\text{"New"}) \cdot P(\text{"York"}) = 0.001 \times 0.0008 = 8 \times 10^{-7}. \quad (11)$$

But the actual joint probability is $0.0007 = 7 \times 10^{-4}$ — about 875 times higher than chance. The PMI ratio is:

$$\frac{P(\text{"New York"})}{P(\text{"New"}) \cdot P(\text{"York"})} = \frac{0.0007}{0.001 \times 0.0008} = \frac{7 \times 10^{-4}}{8 \times 10^{-7}} \approx 875. \quad (12)$$

This is a huge PMI. It tells us that “New” and “York” are strongly associated — whenever you see “New”, there is a much higher than baseline chance the next word is “York” (because “New York” is a fixed expression). The high PMI is informative: it identifies a tight collocation.

Compare this to a non-collocation: “the” and “York”. Both are common (especially “the”), but they have no special tendency to appear adjacently. Their PMI would be close to 1 (or 0 in log form), indicating no association beyond chance.

Constraint on which pairs can be merged: The second token b of a valid pair (a, b) must be word-internal (have **##**), since word-initial tokens can only appear at the start of a word, never in the middle. The first token a can be either word-initial or word-internal. The merged token inherits a ’s status:

- Merging $(n, \text{##}e) \rightarrow ne$: a word-initial token (no **##**).
- Merging $(\text{##}e, \text{##}s) \rightarrow \text{##}es$: a word-internal token.

In the merged token’s name, the **##** of b (if any) is dropped (since the merged token has a ’s status as its prefix).

Token	Computation	freq
l	low (5), lower (2) word-initial	7
n	newest (6) word-initial	6
w	widest (3) word-initial	3
##o	low (5), lower (2) word-internal	7
##w	low (5), lower (2), newest (6) word-internal	13
##e	lower (2), newest ($2 \times 6 = 12$), widest (3) word-internal	17
##r	lower (2) word-internal	2
##s	newest (6), widest (3) word-internal	9
##t	newest (6), widest (3) word-internal	9
##i	widest (3) word-internal	3
##d	widest (3) word-internal	3

Table 8: Individual token frequencies at WordPiece iteration 1.

Pair (a, b)	$\text{freq}(a, b)$	$\text{freq}(a)$	$\text{freq}(b)$	score
(l, ##o)	$5 + 2 = 7$	7	7	$7/49 = 0.143$
(##o, ##w)	$5 + 2 = 7$	7	13	$7/91 = 0.0769$
(##w, ##e)	$2 + 6 = 8$	13	17	$8/221 = 0.0362$
(##e, ##r)	2	17	2	$2/34 = 0.0588$
(n, ##e)	6	6	17	$6/102 = 0.0588$
(##e, ##s)	$6 + 3 = 9$	17	9	$9/153 = 0.0588$
(##s, ##t)	$6 + 3 = 9$	9	9	$9/81 = 0.111$
(w, ##i)	3	3	3	$3/9 = \mathbf{0.333}$
(##i, ##d)	3	3	3	$3/9 = \mathbf{0.333}$
(##d, ##e)	3	3	17	$3/51 = 0.0588$

Table 9: PMI scores at WordPiece iteration 1. Two pairs tie at the maximum 0.333.

8.2 WordPiece Iteration 1

Individual token frequencies in the initial WordPiece segmentation:

Adjacent pairs in the initial segmentation, with PMI scores:

Two pairs tie at the maximum PMI of 0.333: (w, ##i) and (##i, ##d).

Why these two pairs tie: All three of w (word-initial), ##i, and ##d have frequency 3, and they only appear together in “widest”. So all the pairs they form have frequency 3 with denominator $3 \times 3 = 9$, giving the same PMI.

Note: The word-initial w has frequency 3 (only in “widest”), distinct from the word-internal ##w which has frequency 13. If we had treated all w as one token (as in BPE), the pair (w, i) would have score $3/(16 \cdot 3) = 0.0625$, far below the actual WordPiece score. This is a real algorithmic difference, not a notational one.

Lexicographic tie-breaking picks (##i, ##d).

Merge: ##i + ##d $\rightarrow$ ##id.

$V_1^{\text{WP}} = V_0^{\text{WP}} \cup \{\text{##id}\}$, so $|V_1^{\text{WP}}| = 12$.

Updated segmentations:

8.3 WordPiece Iteration 2

Updated token frequencies (only changed ones: ##i, ##d now have frequency 0; ##id has frequency 3):

Pre-token	Freq	Segmentation after WP iteration 1
low	5	(l, ##o, ##w)
lower	2	(l, ##o, ##w, ##e, ##r)
newest	6	(n, ##e, ##w, ##e, ##s, ##t)
widest	3	(w, ##id, ##e, ##s, ##t)

Token	Computation	freq
l	low (5), lower (2)	7
n	newest (6)	6
w	widest (3)	3
##o	low (5), lower (2)	7
##w	low (5), lower (2), newest (6)	13
##e	lower (2), newest (12), widest (3)	17
##r	lower (2)	2
##s	newest (6), widest (3)	9
##t	newest (6), widest (3)	9
##id	widest (3)	3

Adjacent pairs and PMI scores:

Pair (a, b)	$\text{freq}(a, b)$	$\text{freq}(a)$	$\text{freq}(b)$	score
(l, ##o)	7	7	7	$7/49 = 0.143$
(##o, ##w)	7	7	13	$7/91 = 0.0769$
(##w, ##e)	8	13	17	$8/221 = 0.0362$
(##e, ##r)	2	17	2	$2/34 = 0.0588$
(n, ##e)	6	6	17	$6/102 = 0.0588$
(##e, ##s)	9	17	9	$9/153 = 0.0588$
(##s, ##t)	9	9	9	$9/81 = 0.111$
(w, ##id)	3	3	3	$3/9 = \mathbf{0.333}$
(##id, ##e)	3	3	17	$3/51 = 0.0588$

Table 10: PMI scores at WordPiece iteration 2. The pair (w, ##id) now dominates uniquely.

Now (w, ##id) has PMI 0.333, uniquely the maximum.

Merge: w + ##id $\rightarrow$ wid.

The merged token is **word-initial** (no ##) because w was word-initial. $V_2^{\text{WP}} = V_1^{\text{WP}} \cup \{\text{wid}\}$, so $|V_2^{\text{WP}}| = 13$.

Updated segmentations:

8.4 WordPiece Iteration 3

Adjacent pairs and PMI scores:

Maximum is (l, ##o) at 0.143.

Merge: l + ##o $\rightarrow$ lo.

The merged token is **word-initial** (no ##) because l was word-initial. $V_3^{\text{WP}} = V_2^{\text{WP}} \cup \{\text{lo}\}$, so $|V_3^{\text{WP}}| = 14 = V_{\text{target}}$. Training terminates.

Pre-token	Freq	Segmentation after WP iteration 2
low	5	(l, ##o, ##w)
lower	2	(l, ##o, ##w, ##e, ##r)
newest	6	(n, ##e, ##w, ##e, ##s, ##t)
widest	3	(wid, ##e, ##s, ##t)

Pair (a, b)	$\text{freq}(a, b)$	$\text{freq}(a)$	$\text{freq}(b)$	score
(l, ##o)	7	7	7	7/49 = 0.143
(##o, ##w)	7	7	13	7/91 = 0.0769
(##w, ##e)	8	13	17	8/221 = 0.0362
(##e, ##r)	2	17	2	2/34 = 0.0588
(n, ##e)	6	6	17	6/102 = 0.0588
(##e, ##s)	9	17	9	9/153 = 0.0588
(##s, ##t)	9	9	9	9/81 = 0.111
(wid, ##e)	3	3	17	3/51 = 0.0588

Table 11: PMI scores at WordPiece iteration 3. The pair (l, ##o) has the maximum.

8.5 Final WordPiece Vocabulary

$$V^{\text{WP}} = \{ \underbrace{l, n, w}_{\text{initial chars}}, \underbrace{##o, ##w, ##e, ##r, ##s, ##t, ##i, ##d}_{\text{internal chars}}, \underbrace{##id, wid, lo}_{\text{merged tokens}} \} \quad (13)$$

WordPiece does **not** store ordered merge rules. It only stores the vocabulary; tokenization at inference uses greedy longest match.

8.6 Final WordPiece Corpus Segmentation

After all merges, the corpus is segmented as:

Pre-token	Freq	Final WordPiece segmentation
low	5	(lo, ##w)
lower	2	(lo, ##w, ##e, ##r)
newest	6	(n, ##e, ##w, ##e, ##s, ##t)
widest	3	(wid, ##e, ##s, ##t)

8.7 WordPiece Tokenization at Inference

WordPiece uses **greedy longest-match**. Given a word, the first character is matched against word-initial tokens (no ##); subsequent positions are matched against word-internal tokens (with ##).

“newest”:

- Position 0 (word-initial). Try “newest” (not in V), ..., “ne” (not in V), “n” (in V). Emit n.
- Position 1 (word-internal). Try “##ewest” (no), ..., “##e” (yes). Emit ##e.
- Position 2 (word-internal). Try “##west” (no), ..., “##w” (yes). Emit ##w.
- Position 3 (word-internal). Try “##est” (no), “##es” (no), “##e” (yes). Emit ##e.

5. Position 4 (word-internal). Try “##st” (no), “##s” (yes). Emit ##s.
6. Position 5 (word-internal). Try “##t” (yes). Emit ##t.

Final: $\tau_{\text{newest}}^{\text{WP}} = (\text{n}, \text{##e}, \text{##w}, \text{##e}, \text{##s}, \text{##t})$

Six tokens! WordPiece spent its 3 iterations on (##i, ##d), (w, ##id), and (l, ##o) — patterns in “widest” and “low” rather than in “newest”. There was no remaining budget for merges involving “e”, “w”, or “est”.

“low”: $\tau_{\text{low}}^{\text{WP}} = (\text{lo}, \text{##w})$. Two tokens.

“widest”: $\tau_{\text{widest}}^{\text{WP}} = (\text{wid}, \text{##e}, \text{##s}, \text{##t})$. Four tokens.

“newer”: $\tau_{\text{newer}}^{\text{WP}} = (\text{n}, \text{##e}, \text{##w}, \text{##e}, \text{##r})$. Five tokens (no merges help).

9 Side-by-Side Comparison

9.1 Final Vocabularies

Algorithm	Final Vocabulary
BPE	{l, o, w, e, r, n, s, t, i, d, es, est, lo, low} (10 chars + 4 merges = 14 tokens)
WordPiece	{l, n, w, ##o, ##w, ##e, ##r, ##s, ##t, ##i, ##d, ##id, wid, lo} (11 initial + 3 merges = 14 tokens)
UnigramLM	{l, o, w, e, r, n, s, t, i, d, es, est, new, low} (14 tokens after pruning from larger seed)

Table 12: Final vocabularies of the three algorithms trained on the same corpus to $V_{\text{target}} = 14$.

The three algorithms produce *quite different* vocabularies. WordPiece’s looks particularly different because of the ## distinction — nearly all of its 14 tokens are word-internal.

9.2 Tokenization of “newest”

Algorithm	Tokenization of “newest”	# tokens
BPE	(n, e, w, est)	4
WordPiece	(n, ##e, ##w, ##e, ##s, ##t)	6
UnigramLM	(new, est)	2

Table 13: Tokenizations of “newest” across algorithms.

WordPiece produces a 6-token (character-level!) segmentation of “newest”. This is because WordPiece’s high-PMI candidates in this corpus (##id, w+##id, l+##o) were all in “low” or “widest” rather than “newest”. WordPiece spent its 3 iterations elsewhere and never built multi-character tokens for “newest”.

In a larger corpus where “new”, “est”, “##est”, etc. would have high PMI from many word contexts, WordPiece would produce shorter tokenizations. The point is that with only 3 iterations on this tiny corpus, WordPiece’s PMI criterion sent it toward perfect-bigrams elsewhere.

This illustrates a real difference between the criteria: BPE’s raw frequency prioritized the suffix *family* (“es” then “est”) that appears in two words, while WordPiece’s PMI prioritized perfectly-correlated rare bigrams.

9.3 Why “newest” tokenizes so differently

The key insight: **the number of iterations** ($V_{\text{target}} - |V_0|$) is decisive in small examples. BPE had 4 iterations; WordPiece had only 3 (because V_0^{WP} already had 11 tokens vs BPE’s 10). With more iterations, WordPiece would also eventually merge “##es” and “##est”. With $V_{\text{target}} = 16$ instead of 14, WordPiece would likely produce (n, ##e, ##w, ##est).

9.4 Key Properties Comparison

Property	BPE	WordPiece	UnigramLM
Direction	Bottom-up (merge)	Bottom-up (merge)	Top-down (prune)
Merge criterion	Raw frequency	PMI score	Global likelihood
Word-position tokens	Single form per char	Separate ## versions	Single form per char
Initial vocab size	alphabet	alphabet + chars in both positions	Large seed ($\gg V_{\text{target}}$)
Stores at inference	Vocab + ordered rules	Vocab only	Vocab + probabilities
Order-dependent at inference	Yes	No	No
Tokenization at inference	Apply rules sequentially	Greedy longest match	Viterbi on lattice
Probabilistic foundation	None	Local PMI proxy	Global generative model
Stochastic tokenization	Not natively	Not natively	Yes (posterior sampling)
Training speed	Fast	Slower	Slow (inner EM loops)

Table 14: Comparison of the three subword tokenization algorithms.

10 Algorithm Pseudocode

10.1 BPE Training

Algorithm 1 BPE Training

Require: Corpus $\mathcal{W} = \{(w_i, c_i)\}$, target vocabulary size V_{target}

Ensure: Vocabulary V and ordered merge rules $\mathcal{R}$

```

1:  $V \leftarrow$  set of all characters in  $\mathcal{W}$ 
2:  $\mathcal{R} \leftarrow$  empty list
3: Initialize segmentation: each  $w_i$  split into characters
4: while  $|V| < V_{\text{target}}$  do
5:   Count  $\text{freq}(a, b)$  for every adjacent pair  $(a, b)$ 
6:    $(a^*, b^*) \leftarrow \arg \max_{(a, b)} \text{freq}(a, b)$ 
7:    $V \leftarrow V \cup \{a^*b^*\}$ 
8:   Append  $((a^*, b^*) \rightarrow a^*b^*)$  to  $\mathcal{R}$ 
9:   Update segmentations: replace adjacent  $(a^*, b^*)$  with  $a^*b^*$ 
10: end while
11: return  $V, \mathcal{R}$ 

```

10.2 BPE Tokenization at Inference

Algorithm 2 BPE Tokenization

Require: Word w , vocabulary V , ordered merge rules $\mathcal{R}$

Ensure: Tokenization τ

```

1:  $\tau \leftarrow$  characters of  $w$ 
2: for each rule  $r = (a, b) \rightarrow ab$  in  $\mathcal{R}$  in order do
3:   Replace every adjacent occurrence of  $(a, b)$  in  $\tau$  with  $ab$ 
4: end for
5: return  $\tau$ 

```

10.3 WordPiece Training

Algorithm 3 WordPiece Training (with ## notation)

Require: Corpus $\mathcal{W} = \{(w_i, c_i)\}$, target vocabulary size V_{target}

Ensure: Vocabulary V

```

1: Build  $V$  from initial characters with word-position awareness:
2:   For each pre-token, first char  $\rightarrow$  bare token; subsequent chars  $\rightarrow$  ## tokens
3: Initialize segmentation accordingly
4: while  $|V| < V_{\text{target}}$  do
5:   Count  $\text{freq}(t)$  for every token  $t$  in current segmentation
6:   Count  $\text{freq}(a, b)$  for every adjacent pair  $(a, b)$ 
7:    $(a^*, b^*) \leftarrow \arg \max_{(a, b)} \frac{\text{freq}(a, b)}{\text{freq}(a) \cdot \text{freq}(b)}$ 
8:   Compute merged token name: strip ## from  $b^*$  and concatenate to  $a^*$ 
9:   (e.g., ##e + ##s  $\rightarrow$  ##es; n + ##e  $\rightarrow$  ne)
10:   $V \leftarrow V \cup \{\text{merged token}\}$ 
11:  Update segmentations: replace adjacent  $(a^*, b^*)$  with merged token
12: end while
13: return  $V$ 

```

10.4 WordPiece Tokenization at Inference

Algorithm 4 WordPiece Tokenization (Greedy Longest Match)

Require: Word w , vocabulary V

Ensure: Tokenization τ or [UNK]

```

1:  $\tau \leftarrow$  empty list
2:  $i \leftarrow 0$ 
3: while  $i < |w|$  do
4:    $j \leftarrow |w|$ 
5:   target_set  $\leftarrow$  word-initial tokens if  $i = 0$ , else word-internal (##) tokens
6:   while  $j > i$  and corresponding  $w[i : j]$  form  $\notin$  target_set do
7:      $j \leftarrow j - 1$ 
8:   end while
9:   if  $j == i$  then
10:    return [UNK]
11:   end if
12:   Append the corresponding token (with ## prefix if  $i > 0$ ) to  $\tau$ 
13:    $i \leftarrow j$ 
14: end while
15: return  $\tau$ 

```

11 Key Observations

Observation 1: WordPiece’s ## distinction is real, not notational.

Word-initial and word-internal forms of the same character are *different tokens* in WordPiece. They have separate embeddings, separate frequency counts, and can only merge with appropriately

positioned partners. This is not just a display convention — it is a fundamental aspect of the algorithm that affects which merges get chosen.

In our example, the word-initial `w` has frequency 3 (only in “widest”) while the word-internal `##w` has frequency 13. Treating them as one token would give (w, i) a much lower PMI score and would change which merge gets selected first.

Observation 2: BPE merges what is common; WordPiece merges what is informative.

BPE selected (e, s) at iteration 1 because it occurs 9 times — the highest raw frequency. WordPiece selected $(##i, ##d)$ at iteration 1 because `##i` and `##d` are individually rare (3 each) but always co-occur, giving a high PMI of 0.333. The difference between these criteria becomes more visible the smaller the corpus.

Observation 3: Both algorithms are bottom-up and irreversible.

Once $(a, b) \rightarrow ab$ is committed, it cannot be undone. Subsequent decisions are evaluated on the post-merge corpus. This is fundamentally different from UnigramLM, where the EM step can re-evaluate any token’s utility.

Observation 4: BPE stores rules; WordPiece stores only the vocabulary.

BPE at inference applies its ordered merge rules sequentially. The order matters. WordPiece at inference uses only the vocabulary via greedy longest match. The vocabulary alone determines tokenization.

Observation 5: Out-of-vocabulary handling differs.

- BPE: characters always present, so falls back to character-level segmentation.
- WordPiece: if a character doesn’t appear in the vocabulary at its position, returns `[UNK]` for the whole word. In practice rare because the character vocabulary is comprehensive.
- UnigramLM: Viterbi always succeeds when all characters are in V .

Observation 6: Byte-level BPE eliminates the OOV character problem.

Modern BPE (GPT-2 onward) operates on UTF-8 bytes. The base vocabulary is 256 byte values plus learned merges. Any input string can be tokenized into bytes, eliminating OOV characters entirely. This is why LLaMA and GPT tokenizers never produce `[UNK]`.

Observation 7: For our small corpus, WordPiece produced a much sparser tokenization of “newest”.

WordPiece’s tokenization of “newest” has 6 tokens versus BPE’s 4. This is partly because of WordPiece’s `##` distinction reducing the merge budget (3 iterations vs 4) and partly because its PMI criterion sent the merges toward “widest”-specific patterns rather than the “newest”/“widest” shared suffix that BPE captured.

In a real-world corpus of millions of words, both algorithms have many iterations available, and the qualitative differences come from which subword patterns each criterion prefers — not from the tiny iteration-budget differences seen here.

Observation 8: When in doubt, match what your base model uses.

If extending or fine-tuning a model that used BPE (LLaMA, GPT), use BPE. If extending WordPiece (BERT, mBERT), use WordPiece. If extending UnigramLM (XLM-R, Glot500), use UnigramLM.

12 Why This Matters for the Coptic-Syriac Project

For our Coptic-Syriac extension of Glot500, we use UnigramLM via SentencePiece because Glot500 itself uses UnigramLM. This tutorial on BPE and WordPiece is included for completeness.

If you were instead extending LLaMA or GPT-2, you would use BPE. Relevant tools:

- HuggingFace `tokenizers` library with `BpeTrainer`.
- OpenAI `tiktoken` for byte-level BPE inference.
- SentencePiece with `model_type=bpe` (this is what LLaMA used).

If you were extending BERT or mBERT, you would use WordPiece. Relevant tools:

- HuggingFace `tokenizers` library with `WordPieceTrainer`.
- The BERT tokenizer in `transformers`.

The methodology of vocabulary extension (training auxiliary tokenizer, merging vocabularies, embedding initialization, continued pretraining) is largely independent of which tokenization algorithm is used.

The main implementation pitfall for WordPiece: when extending a WordPiece-based model with new tokens, you must respect the `##` distinction. A new token that should be word-internal must enter the vocabulary as `##X`, not as the bare `X`. Embedding initialization methods (Mean, Align, FOCUS) must also be applied separately to the word-initial and word-internal forms; treating them as the same token will give wrong results.

References

- [1] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of ACL*, 2016.
- [2] Mike Schuster and Kaisuke Nakajima. Japanese and Korean Voice Search. In *Proceedings of ICASSP*, 2012.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL*, 2019.
- [4] Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proceedings of EMNLP: System Demonstrations*, 2018.
- [5] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In *Proceedings of ACL*, 2018.